

# *Cheat Sheet for React*

Things I learned while working on the [Tic-Tac-Toe](#) Tutorial

## Content

|                                     |    |
|-------------------------------------|----|
| React Project Structure (Vite)..... | 3  |
| Folders .....                       | 3  |
| Key Files .....                     | 3  |
| Summary.....                        | 3  |
| Explanations Build Tools .....      | 4  |
| React Terms and syntax .....        | 5  |
| React Best Practices.....           | 9  |
| Naming Conventions .....            | 9  |
| immutability.....                   | 10 |
| Re-renders .....                    | 10 |

## React Project Structure (Vite)



### Folders

- `node_modules/` Contains all installed third-party libraries and dependencies (e.g. React itself)
- `public/` Contains static assets (images, favicons) that are served directly to the browser.
- `src/` folder where the application code is written (Components, CSS, Logic)

### Key Files

- `index.html` The entry point for the Browser. It contains the empty container `<div id="root"></div>` where the React app will be injected.
- `Main.jsx` The JavaScript entry point. Connects React to the HTML. Finds the `#root` and renders the `<App />` components inside it.
- `App.jsx` Main component of the Application. This is where the UI structure and content of the Website is defined.
- `Package.json` The Configuration file. It lists the project dependencies (libraries) and defines scripts (like `npm run dev` to start the server)
- `Vite.config.js` Configuration settings specially for the Vite build tool

### Summary

The Browser loads `index.html` → which loads `src/main.jsx` → which renders `src/App.jsx`

## Explanations Build Tools

Because browsers cannot directly understand the code that's written in a modern React project. We use special syntax like JSX (writing HTML inside JavaScript) and split the code into many separate files for better organization. A build Tool acts as a bridge between the developer code and the browser.

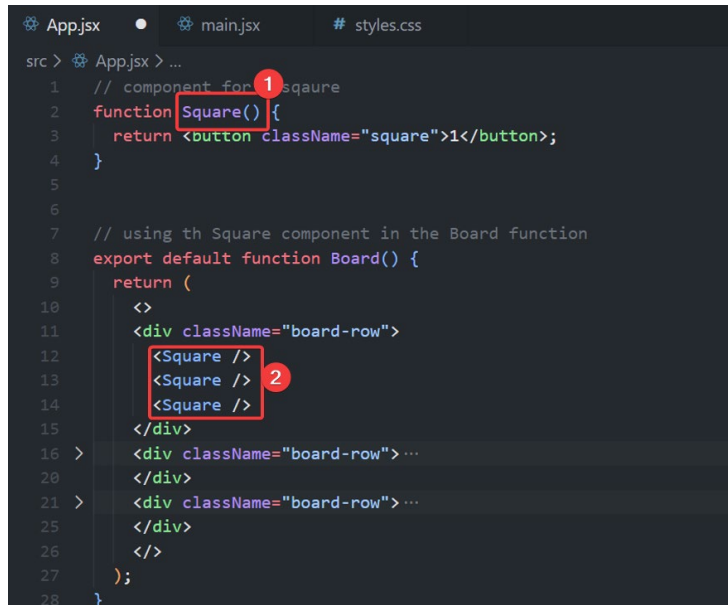
Main jobs:

- **Converting:** Converting JSX and modern JavaScript into standard JavaScript that every browser can read.
- **Building:** Merging hundreds of small code files into a few optimized files for the website to load.
- **Development Server:** Providing a local web server that lets the developer see the changes instantly.

Example Vite: Is a modern frontend tool designed to be significantly faster than other tools like Create React App. Another benefit of using Vite is that it provides a pre-configuration template for React, so the complex settings are already configured.

## React Terms and syntax

**Component** – piece of reusable code that represents a part of a user interface. Components are used to render, manage and update the UI elements in the Application.

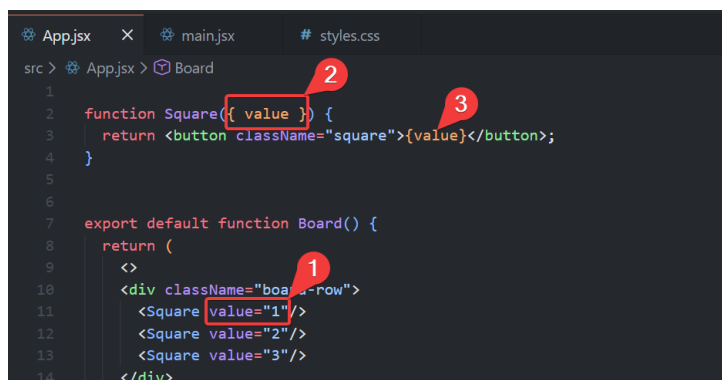


```
src > App.jsx • main.jsx # styles.css
src > App.jsx > ...
1 // component for square
2 function Square() {
3   return <button className="square">1</button>;
4 }
5
6
7 // using th Square component in the Board function
8 export default function Board() {
9   return (
10     <>
11       <div className="board-row">
12         <Square />
13         <Square />
14         <Square />
15       </div>
16     <div className="board-row"> ...
17   </div>
18   <div className="board-row"> ...
19 </div>
20 );
21 };
```

*Example of reusing Square Component in Board function*

**JSX-Element** – Combination of JavaScript code and HTML tags.

**props** – Arguments passed into React components. They're passed through HTML attributes.



```
src > App.jsx × main.jsx # styles.css
src > App.jsx > Board
1
2 function Square({ value }) {
3   return <button className="square">{value}</button>;
4 }
5
6
7 export default function Board() {
8   return (
9     <>
10       <div className="board-row">
11         <Square value="1"/>
12         <Square value="2"/>
13         <Square value="3"/>
14       </div>
15     </div>
16   );
17 };
```

Example of passing {value} through HTML attribute<sup>[1]</sup> from parent-class (Board) to the child-class<sup>[2]</sup> (Square) and rendering it inside the HTML element<sup>[3]</sup>.

state – In a State you store a Value that belongs to a component. For example, making a component remember something (e.g. that it got clicked). When the state changes the component re-renders.



```
src > App.jsx > Board
1  import { useState } from "react";
2
3  // components for a square
4  function Square() {
5
6    // value - stores the Value
7    // setValue - function to change the value
8    // null - passed a initial value for this state variable
9    const [value, setValue] = useState(null);
10
11
12    // handle click of the Button
13    function handleClick() {
14      setValue('X');
15    }
16
17    // Loading the Button in HTML
18    return (
19      <button className="square" onClick={handleClick}>{value}</button>
20    );
21  }
22
```

Example, every first time the `Board()` function calls a single square, react sets the starting Value (state) of each square to `null`<sup>[1]</sup> . (because it is the initial render of the website and nothing else (like `setValue`) was called.

When the user clicks the square, the whole square function gets re-rendered again.

And because after clicking the square, the button calls the `handleClick()` function which uses `setValue()`<sup>[2]</sup>, React notices that and doesn't set the value `Null`, instead it stores the value from `setValue()` in `value`<sup>[3]</sup> which gets rendered in the button text.

() => – This syntax is an Arrow function; it's a shorter way of defining functions.

Example

Before:

```
hello = function() {  
  return "Hello World!";  
}
```

Try it Yourself »

Example

Arrow Functions Return Value by Default:

```
hello = () => "Hello World!";
```

Try it Yourself »

Example before VS after.

```
<Square value={squares[0]} onClick={handleClick(0)} />
```

**The Problem:** When the board gets rendered, it instantly calls the `handleClick(0)` function. This causes the board to change its state because `handleClick(0)` calls `setSquares`. As a result, the board re-renders and calls the `handleClick(0)` function again, it's an infinite loop of re-renders.

```
<div className="board-row">  
  <Square value={squares[0]} onClick={() => handleClick(0)} />  
  // ...  
</div>
```

**The Solution:** The arrow function acts like a brake. When the board renders, it says: "Here is a function, but don't call it now, only call it when the user clicks."

### Excuse function calling →

What is the difference between these two:

```
<Square value={squares[0]} onClick={handleClick} />  
  
<Square value={squares[0]} onClick={handleClick(0)} />
```

**Line 1**, without parentheses, is passing a function **reference**. It Tells React:

"Here is a function please call it later when the user clicks"

As a result, it does not run during render.

**Line 2**, with parentheses, it is calling the function immediately (**Call Operator**),

Javascript executes this function while it renders the board, which causes the infinite loop explained above.

**It is a Rule in JavaScript which causes this.**

Short if-else statement – This is the short Form of writing an if-else statement

```
let player = (xIsNext ? "X" : "O");
```

### Syntax

```
variable = (condition) ? expressionTrue : expressionFalse;
```

Spread syntax (...) – In the example below, the three dots perform three steps:

1. It creates a new array
2. It copies (spreads) all existing items from the old history array into the new one.
3. it adds the “nextSquares” item to the end of the new array

Why? React needs a new array copy to recognize that data has changed and to trigger a re-render. SO the old array gets never modified directly.

```
setHistory([...history, nextSquares]);
```

Create a new array based on the old array with addition of the new data.

.map() Method – is used in JS to transform one array into another. For example transform the history of data into a list of visual buttons for the user.

Key property – key acts like a unique ID card for every item in a list, It gibes each component a stable identity.

It is needed because when a list changes (items are added, removed, re-ordered), React needs to know exactly which item is which.

```
return (  
  <li key={move}>  
    <button onClick={() => jumpTo(move)}>{description}</button>  
  </li>  
);
```

In this code we use the index of the move as the Key, this works because moves in Tic-Tac-Toe are never reordered or inserted in the middle.

# React Best Practices

## Naming Conventions

Build-in Components (e.g. <button>): The naming is strict. You must use onClick because React directly uses the browser's click event.

Custom components (e.g. <Square>, in the code above) The name is up to you. You can name the prop whatever you want (e.g. onSuperSmash). However, there is a standard convention to keep code readable:

- **onEvent** for **props** (e.g., onClick): Describes *when* something happens.
- **handleEvent** for **functions** (e.g., handleClick): Describes *what* the function does in response.

AI-explanation

```
<div className="board-row">
  <Square value={squares[0]} onClick={() => handleClick(0)} />
  <Square value={squares[1]} onClick={() => handleClick(1)} />
  <Square value={squares[2]} onClick={() => handleClick(2)} />
</div>
```

When the Square gets clicked, the function handles this Click and does something in response.

## immutability

```
21     function handleClick(i) {  
22         ⚡ const nextSquares = squares.slice();  
23         nextSquares[i] = "X";  
24         setSquares(nextSquares);  
25     }
```

In this `handleClick` function we call `.slice()` to create a copy of the `squares` array instead of modifying the existing one. Why?

There are two ways of changing data:

> mutate (directly changing the data's value)

```
const squares = [null, null, null, null, null, null, null, null, null];  
squares[0] = 'X';  
// Now `squares` is ['X', null, null, null, null, null, null, null, null];
```

> replacing the data with a new copy which has the change

```
const squares = [null, null, null, null, null, null, null, null, null];  
const nextSquares = ['X', null, null, null, null, null, null, null, null];  
// Now `squares` is unchanged, but `nextSquares` first element is 'X' rather than `null`
```

What are the benefits:

Immutability makes complex features easier to implement, for example in an app it's a common feature to be able to undo or redo actions. Avoiding direct data mutation lets you keep previous versions of the data which can be reused later.

## Re-renders

Although re-rendering is not noticeable to the user, it is important to try avoiding it for performance reasons.